

**ΥΛΟΠΟΙΗΣΗ
ΣΥΣΤΗΜΑΤΟΣ
ΑΠΟΣΤΟΛΕΑ-ΔΕΚΤΗ ΣΕ
VERILOG ΜΕ ΧΡΗΣΗ
ΠΡΩΤΟΚΟΛΛΟΥ UART
ΚΑΙ ΠΡΟΒΟΛΗ ΣΕ
ΟΘΟΝΗ ΤΕΣΣΑΡΩΝ LED
7-ΤΜΗΜΑΤΩΝ**

**ΓΚΑΤΖΗ ΚΩΝΣΤΑΝΤΙΝΑ
ΠΑΠΑΚΩΣΤΑΣ ΚΩΝΣΤΑΝΤΙΝΟΣ**

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Εργασία στο μάθημα Ψηφιακά Συστήματα Χαμηλής Λογικής HW I

Υπεύθυνος καθηγητής: Σκεντόπουλος

Σύνταξη: Γκατζή Κωνσταντίνα, Παπακώστας Κωνσταντίνος

A.E.M: 10037 , 10127

Ημερομηνία: Θεσσαλονίκη, 20/01/2023



Περιεχόμενα

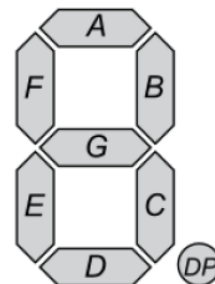
ΜΕΡΟΣ Α' - 7 SEGMENT DISPLAY DRIVER.....	4
1) Οθόνη 7-τμημάτων και αποκωδικοποιητής	4
2) Οδήγηση 4 ανόδων.....	5
3) Λειτουργικός Έλεγχος	6
4) Σχηματικά.....	7
ΜΕΡΟΣ Β' – UART.....	9
1) Ελεγκτής Baud Rate	9
2) Υλοποίηση UART Αποστολέα.....	10
3) Υλοποίηση UART Δέκτη	11
4) Λειτουργικός Έλεγχος	12
5) Σχηματικά.....	14
ΜΕΡΟΣ Γ' – ΣΥΝΕΝΩΣΗ.....	16
ΜΕΡΟΣ Δ' – ΚΩΔΙΚΟΠΟΙΗΤΗΣ/ΑΠΟΚΩΔΙΚΟΠΟΙΗΤΗΣ.....	18
1) Κωδικοποιητής Gray	18
2) Αποκωδικοποιητής Gray	18
3) Λειτουργικός Έλεγχος	19
4) Σχηματικά.....	21
ΣΥΝΔΕΣΜΟΙ EDAPLAYGROUND	22
ΣΤΟΙΧΕΙΑ ΦΟΙΤΗΤΩΝ.....	23

ΜΕΡΟΣ Α' - 7 SEGMENT DISPLAY DRIVER

1) Οθόνη 7-τμημάτων και αποκωδικοποιητής

Η οθόνη 7 τμημάτων αποτελεί την απλούστερη μορφή αναπαράστασης ενός αλφαριθμητικού ψηφίου. Το κάθε ψηφίο αποτελείται από τα 7 συν 1 (δεκαδικό σημείο DP) σήματα A, B, C, D, E, F, G και DP, όπως φαίνεται στην Εικόνα 1.

Για την αποκωδικοποίηση των 7 συν 1 τμημάτων υλοποιήσαμε ένα συνδυαστικό αποκωδικοποιητή μέσω της συνάρτησης *LEDdecoder*, ο οποίος έχει ως είσοδο το ψηφίο ή χαρακτήρα που επιθυμούμε να εμφανίσουμε σε κώδικα BCD (*wire char [3:0]*), και παράγει ως έξοδο τις τιμές οδήγησης των 8 τμημάτων (*reg LED [6:0]*). Για τη σωστή λειτουργία του αποκωδικοποιητή (0 = αναμμένο, 1 = σβηστό), για κάθε θέση του καταχωρητή δημιουργήθηκε ξεχωριστά η συνάρτηση Boole σύμφωνα με την αποκωδικοποίηση που προτείνεται από το βιβλίο του μαθήματος ($G \rightarrow \text{LED}[0], \dots, \text{DP} \rightarrow \text{LED}[7]$ και $A \rightarrow \text{char}[3], \dots, D \rightarrow \text{char}[0]$).



Πιο συγκεκριμένα:

ΕΙΚΟΝΑ 1: ΟΘΟΝΗ 7 ΤΜΗΜΑΤΩΝ

- $a = \overline{AC} + \overline{ABD} + \overline{BCD} + \overline{ABC} + \overline{ABCD}$
- $b = \overline{AB} + \overline{ACD} + \overline{ACD} + \overline{ABC}$
- $c = \overline{AB} + \overline{AD} + \overline{BCD} + \overline{ABC}$
- $d = \overline{ACD} + \overline{ABC} + \overline{BCD} + \overline{ABC} + \overline{ABCD}$
- $e = \overline{ACD} + \overline{BCD} + \overline{ABD} + \overline{ABCD}$
- $f = \overline{ABC} + \overline{ACD} + \overline{ABD} + \overline{ABC} + \overline{ABCD}$
- $g = \overline{ACD} + \overline{ABC} + \overline{ABC} + \overline{ABC} + \overline{ABCD} + \overline{ABC}$
- $dp = 1$

Τέλος, σημειώνεται πως υποθέσαμε ότι ο χαρακτήρας '-' αναπαρίσταται από το δυαδικό 1010 ενώ ο χαρακτήρας 'F' από το 1011.

Στις εικόνες 2 και 3 φαίνεται η αποκωδικοποίηση για κάθε αριθμό από το 0 έως το 9, την παύλα και τον χαρακτήρα f.

	0	10	20	30	40	50
bcd[3:0]	0	1	10	11	100	101
seg[6:0]	1	1001111	10010	110	1001100	100100

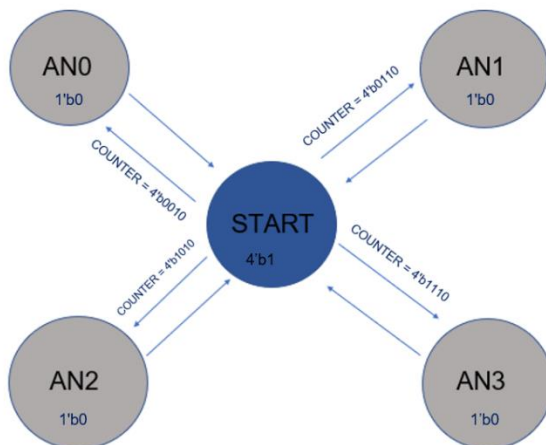
Εικόνα 2: Αποκωδικοποίηση 7 τμημάτων για τους αριθμούς 0 έως 5

	60	70	80	90	100	110
bcd[3:0]	110	111	1000	1001	1010	1011
seg[6:0]	100000	1111	0	100	1111110	111000

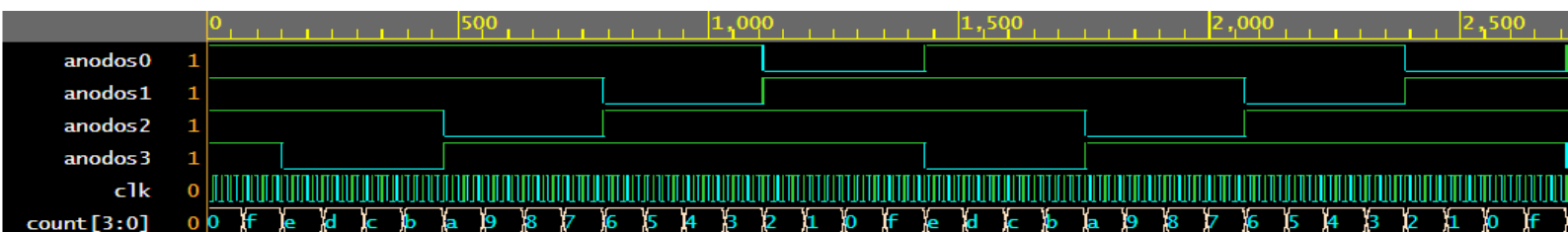
Εικόνα 3: Αποκωδικοποίηση 7 τμημάτων για τους αριθμούς 6 έως 9 και τους χαρακτήρες

2) Οδήγηση 4 ανόδων

Για την ορθή οδήγηση των 4 ανόδων υλοποιήθηκε η συνάρτηση *Anodoi*. Η συνάρτηση αυτή χρησιμοποιεί έναν μετρητή και σε συγκεκριμένες τιμές αυτού η άνοδος an_x οδηγείται στο λογικό 0 (ανάβει) όπως υποδεικνύεται από την εργασία. Πρακτικά ο μετρητής χρησιμοποιείται ως FSM του οποίου το σχεδιάγραμμα απεικονίζεται στην εικόνα 4. Έτσι στην τιμή 1110 η an_3 οδηγείται στο 0, στο 1010 η an_2 , στο 0110 η an_1 , και στο 0010 η an_0 . Τα παραπάνω γίνονται εμφανή στην εικόνα 5.

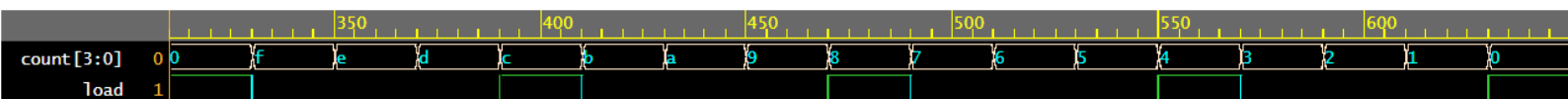


Εικόνα 4: Σχηματικό του FSM οδήγησης των ανόδων.



Εικόνα 5: Οδήγηση των ανόδων.

Επιπλέον όπως ζητείται από την εργασία τα δεδομένα φορτώνονται στον καταχωρητή char σε συγκεκριμένες τιμές του μετρητή έτσι ώστε να είναι έτοιμα για προβολή στις 4 οθόνες του συστήματος μας. Πιο συγκεκριμένα τα δεδομένα φορτώνονται κατά τη θετική άνοδο του σήματος load στις τιμές του μετρητή 12, 8, 4 και 0.



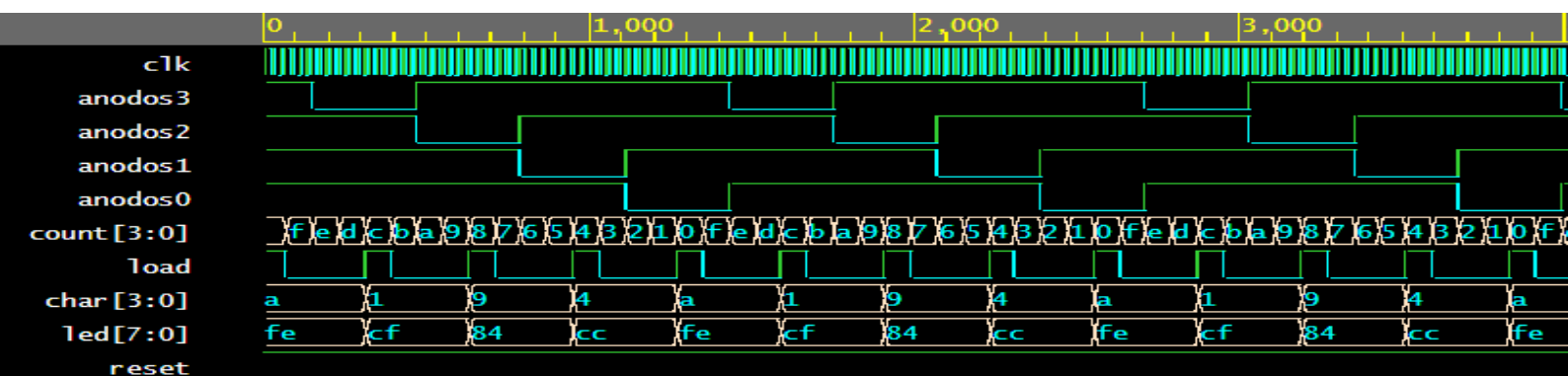
Εικόνα 6: Φόρτωση χαρακτήρων στον καταχωρητή.

Για την διευκόλυνση μας δημιουργήσαμε μια επιπλέον συνάρτηση *counter* που αναπαριστά τον μετρητή που προαναφέραμε. Η συνάρτηση έχει ως ορίσματα τα *clock* και *reset* του συστήματος μας και η έξοδος της είναι ένας καταχωρητής με τις τιμές του μετρητή σε κώδικα BCD. Ο καταχωρητής ξεκινά να μετρά από την τιμή f (15 στο δεκαδικό) και στην συνέχεια ελαττώνεται κατά 1 έως ότου φτάσει στο 0 και αυτό επαναλαμβάνεται αενάως. Με το σήμα *reset* ο μετρητής αρχικοποιείται στο 0 σε οποιαδήποτε στιγμή και αν βρίσκεται και συνεχίζει από εκεί τη λειτουργία του.

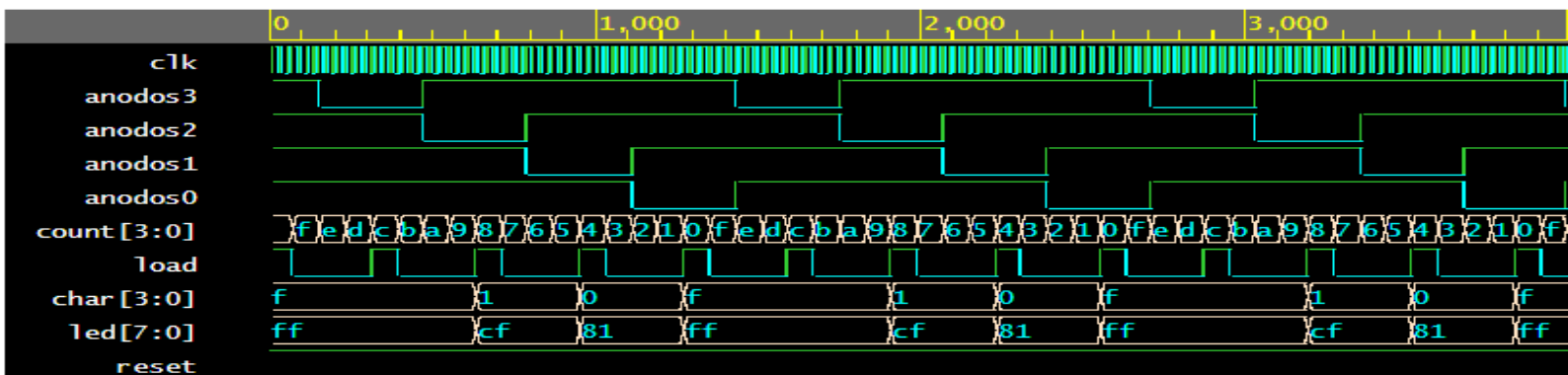
3) Λειτουργικός Έλεγχος

Για τον λειτουργικό έλεγχο υλοποιήθηκε η συνάρτηση *Meros_A*. Στο εσωτερικό της υλοποιήθηκε ρολόι περιόδου 20ns όπως απαιτείται από την εκφώνηση. Έπειτα εισαγάγαμε μια μνήμη για τη αποθήκευση των χαρακτήρων που πρέπει να εκτυπώσουμε και σε κάθε θετική άνοδο του σήματος *load* ένας-ένας χαρακτήρας κωδικοποιούνται σε κώδικα BCD όπως εξηγήθηκε παραπάνω και διοχετεύεται στον καταχωρητή *char* ο οποίος με τη σειρά του αποτελεί είσοδο στην συνάρτηση αποκωδικοποίησης για τις οθόνες. Το πρόγραμμα τελειώνει όταν εμφανιστούν στην οθόνη και οι 4 χαρακτήρες.

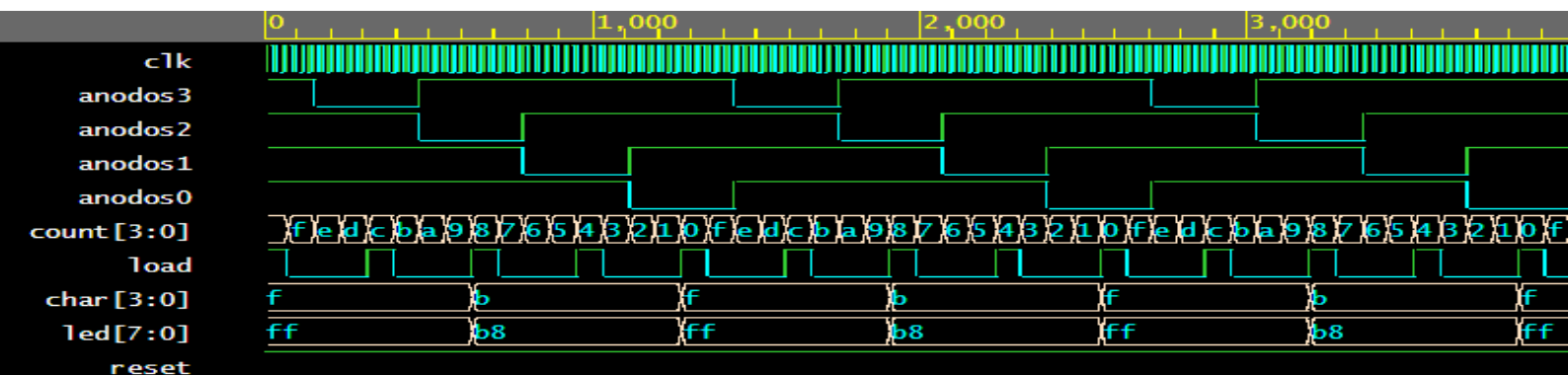
Παρακάτω παρατίθενται εικόνες που επιβεβαιώνουν την ορθή λειτουργία του κυκλώματος, τους χρονισμούς καθώς και τις σχέσεις μεταξύ σημάτων κατά την προβολή των μηνυμάτων '-194' (εικόνα 7), ' 10' (εικόνα 8), 'FFFF' (εικόνα 9).



Εικόνα 7: Εμφάνιση του μηνύματος «-194».



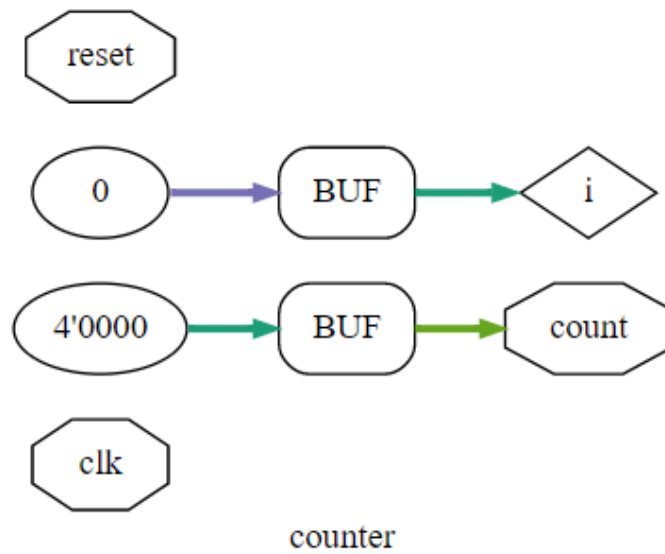
Εικόνα 8: Εμφάνιση του μηνύματος « 10».



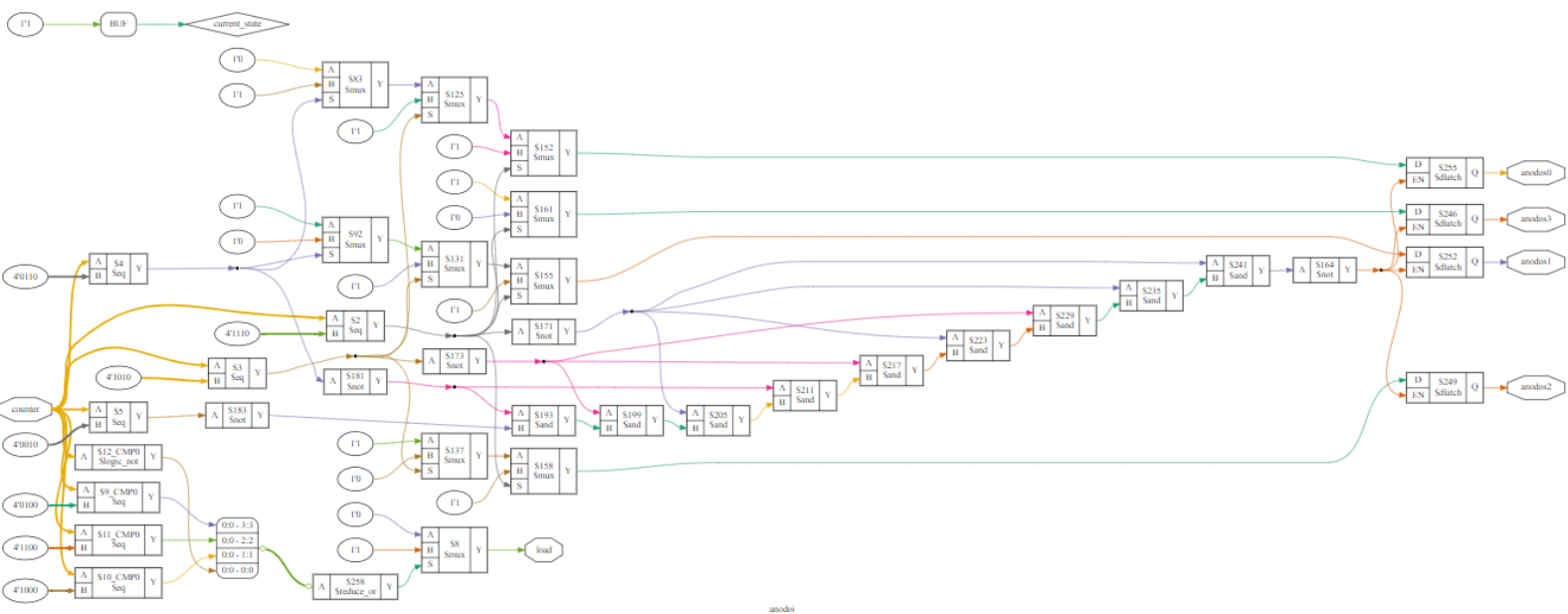
Εικόνα 9: Εμφάνιση του μηνύματος «ffff».

4) Σχηματικά

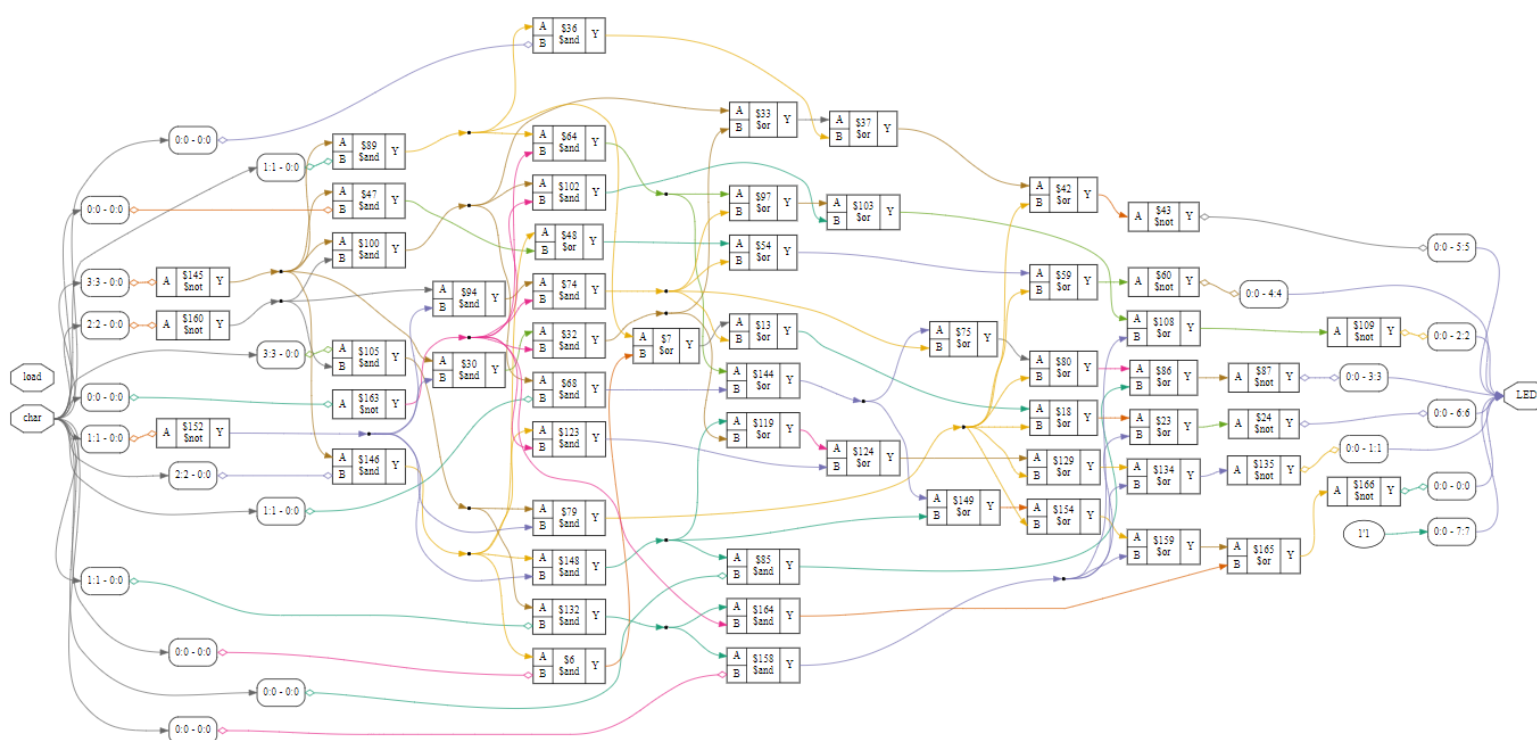
Τα σχήματα που προέκυψαν από τις συναρτήσεις είναι τα εξής:



Εικόνα 10: Σχηματική αναπαράσταση του module counter.



Εικόνα 11: Σχηματική αναπαράσταση του module anodoi.



Εικόνα 12: Σχηματική αναπαράσταση του module LED_Decoder.

ΜΕΡΟΣ Β' – UART

Στο μέρος Β της εργασίας ζητείται να υλοποιήσουμε ένα σύστημα Γενικού Ασύγχρονου Δέκτη Αποστολέα (UART). Το σύστημα καλείται να μεταφέρει μια αλληλουχία τεσσάρων διαφορετικών συμβόλων από τον αποστολέα προς τον δέκτη. Η μεταφορά γίνεται ανά δύο σύμβολα. Για την ορθή επικοινωνία μεταξύ των δύο, το μήνυμα κωδικοποιείται ως εξής: Αρχικά στέλνεται το start bit (0'b0) για να γίνει γνωστή στον δέκτη η αρχή του μηνύματος. Στη συνέχεια στέλνονται σειριακά τα 8 bit των δύο συμβόλων προς αποστολή. Έπειτα για την διασφάλιση της ορθής λήψης των bits από τον δέκτη, ο αποστολέας στέλνει ένα bit που συμβολίζει τον αριθμό των λογικών 1 στο προς αποστολή μήνυμα (1'b0 αν το μήνυμα περιέχει άρτιο αριθμό λογικών 1 και 1'b1 αν περιέχει περιττό). Τέλος στέλνεται το end bit (1'b1) που σηματοδοτεί το τέλος της μετάδοσης.

1) Ελεγκτής Baud Rate

Για την σωστή επικοινωνία μεταξύ αποστολέα και δέκτη, οι δύο προσυμφωνούν την ταχύτητα επικοινωνίας σε μονάδες Baud Rate. Στον παρακάτω πίνακα παρουσιάζονται οι ταχύτητες επικοινωνίας και η σχετική κωδικοποίηση τους.

BAUD_SEL	Baud Rate
000	300 bits/sec
001	1200 bits/sec
010	4800 bits/sec
011	9600 bits/sec
100	19200 bits/sec
101	38400 bits/sec
110	57600 bits/sec
111	115200 bits/sec

Πίνακας 1 : Πίνακας Baud Rate

Σε αυτό το κομμάτι της εργασίας υλοποιήθηκε η συνάρτηση *baud_controller* στόχος της οποίας είναι να παρέχει το κατάλληλο σήμα δειγματοληψίας ανάλογα με το επιλεγμένο baud rate. Χρησιμοποιώντας ρολόι των 50MHz υπολογίστηκαν οι μέγιστες τιμές που μπορεί να λάβει ένας μετρητής ανάλογα με το κάθε baud rate. Οι τιμές αυτές παρουσιάζονται στον παρακάτω πίνακα μαζί με τα σχετικά σφάλματα.

Baud_Sel	Baud Rate	50M/(Baud_Rate*16)	Μέγιστη Τιμή	Σχετικό Σφάλμα
000	300 bits/sec	10416.66	10417	-0.003%
001	1200 bits/sec	2604.16	2604	0.006%
010	4800 bits/sec	651.04	651	0.006%
011	9600 bits/sec	325.52	326	-0.15%
100	19200 bits/sec	162.76	163	-0.15%
101	38400 bits/sec	81.38	81	0.47%
110	57600 bits/sec	54.25	54	0.46%
111	115200 bits/sec	27.12	27	0.47%

Πίνακας 2 : Baud Rate

Παρατηρούμε πως όσο αυξάνεται η ταχύτητα επικοινωνίας τόσο μεγαλώνει το σχετικό σφάλμα στο Baud Rate.

Έτσι η συνάρτηση *baud_controller* δέχεται ως ορίσματα τα *clock* και *reset* του συστήματος μας καθώς και τον προκαθορισμένο ρυθμό επικοινωνίας κωδικοποιημένο σε 3 bit (*baud_sel*) και παράγει στην έξοδο *sample_ENABLE* έναν παλμό ο οποίος έχει περίοδο x κύκλους ρολογίου ανάλογα με τον προεπιλεγμένο ρυθμό επικοινωνίας.

2) Υλοποίηση UART Αποστολέα

Υλοποιούμε τώρα έναν UART αποστολέα. Η συνάρτηση *uart_transmitter* έχει ορίσματα τα:

- *Tx_DATA*: τα προς εμφάνιση δεδομένα σε καταχωρητή 8 θέσεων
- *baud_select*: ο προεπιλεγμένος ρυθμός επικοινωνίας κωδικοποιημένος σε 3 bit
- *Tx_EN*: σήμα ενεργοποίησης του αποστολέα
- *Tx_WR*: σήμα παραλαβής δεδομένων
- *clk, reset*: το ρολόι του συστήματος

και οι έξοδοι είναι:

- *Tx_BUSY*: σήμα κατάστασης του αποστολέα
- *TxD*: το κωδικοποιημένο μήνυμα όπως εξηγήθηκε στην αρχή της ενότητας

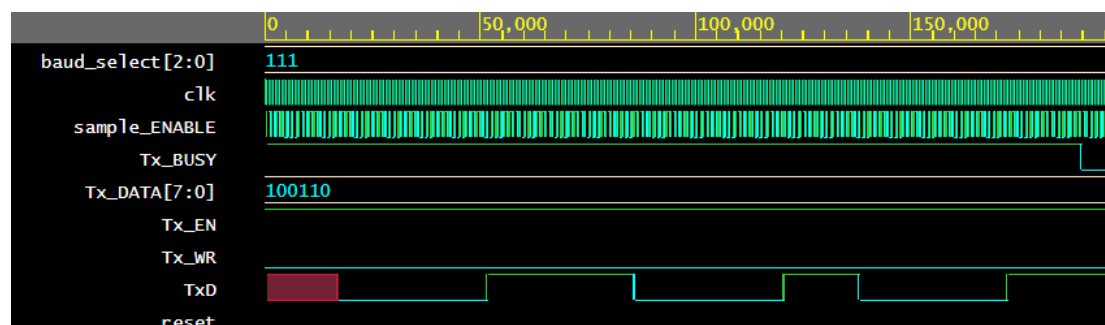
Αρχικά ένα το σήμα *Tx_EN* είναι στο **λογικό 1**, ενεργοποιείται ο αποστολέας θέτοντας απευθείας την έξοδο *Tx_BUSY* σε **λογικό 1** υποδηλώνοντας ότι βρίσκεται σε κατάσταση αποστολής.

Στη συνέχεια, προσπελάζει ένα προς ένα τα στοιχεία του καταχωρητή *Tx_DATA*. Εάν το πλήθος των **λογικών 1** είναι περιττό θέτει την τιμή **1** στην μεταβλητή parity που ορίσαμε εμείς για διευκόλυνση.

Έπειτα καλείται η συνάρτηση του προηγούμενου ερωτήματος και κάθε 16 κύκλους του νέου ρολογίου οδηγείται η έξοδος *TxD* σύμφωνα με την κωδικοποίηση που αναφέραμε στην αρχή της ενότητας.

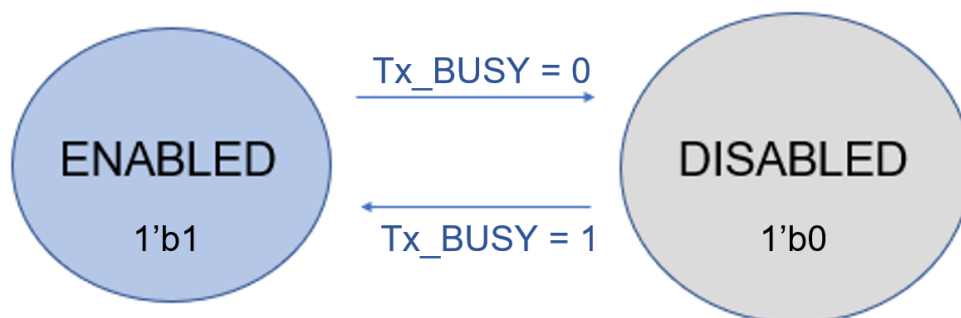
Τέλος αφού ολοκληρωθεί η αποστολή των 11 bit (start bit, δεδομένα, parity bit, end bit) η έξοδος *Tx_BUSY* τίθεται στο **0** και δίνεται εντολή στο κύκλωμα να απενεργοποιήσει τον αποστολέα θέτοντας την τιμή **0** στην είσοδο *Tx_EN*.

Στην παρακάτω εικόνα απεικονίζεται η παραπάνω διαδικασία κατά την αποστολή του μηνύματος '26' με ρυθμό αποστολής 115200 bits/sec.



Εικόνα 13: UART Αποστολέας – Αποστολή του μηνύματος '26'??

Τέλος σημειώνεται πως ο UART αποστολέας αποτελεί ουσιαστικά μια μηχανή FSM δύο καταστάσεων το σχηματικό της οποίας παρουσιάζεται στην εικόνα 14.



Εικόνα 14: Σχηματικό FSM UART Αποστολέα.

3) Υλοποίηση UART Δέκτη

Υλοποιούμε τώρα έναν UART Δέκτη. Η συνάρτηση *uart_receiver* έχει ορίσματα τα:

- *RxD*: τα bit που λαμβάνει ο δέκτης μέσω του καναλιού επικοινωνίας
- *baud_select*: ο προεπιλεγμένος ρυθμός επικοινωνίας κωδικοποιημένος σε 3 bit
- *RX_EN*: σήμα ενεργοποίησης του δέκτη
- *clk, reset*: το ρολόι του συστήματος

και οι έξοδοι είναι:

- *Rx_DATA*: καταχωρητής με τα ληφθέντα δεδομένα
- *Rx_ERROR*: σήμα ένδειξης λάθους στα bit (άρτιος η περιττός αριθμός λογικών 1)
- *Rx_FERROR*: σήμα ένδειξης λάθους στον επιλεγμένο ρυθμό επικοινωνίας
- *Rx_VALID*: σήμα ένδειξης ορθής λήψης των δεδομένων

Αρχικά ορίζονται και αρχικοποιούνται οι βοηθητικές μεταβλητές:

- *countbits*: για την μέτρηση των bit που λαμβάνουμε
- *ones*: για την καταμέτρηση των λογικών 1
- *error1 & error2*: σφάλμα στο parity του πρώτου και του δεύτερου πακέτου
- *baud_check*: έλεγχος για την ορθή επιλογή ρυθμού επικοινωνίας
- *times*: υποδεικνύει την μετάδοση του $1^{ου}$ (=0) ή $2^{ου}$ (=1) πακέτου

Όταν το σήμα *RX_EN* είναι στο λογικό 1, ενεργοποιείται ο δέκτης.

Με κάθε αλλαγή της εισόδου *RxD* η τιμή αυτής καταχωρείται στον καταχωρητή bits ενώ ταυτόχρονα αυξάνεται η τιμή του *baud_check* η οποία πρέπει να φτάσει την τιμή 10 δηλαδή να έχουν μεταδοθεί 11 bits πληροφορίας. Εναλλακτικά συμπεραίνω πως έχω σφάλμα στον ρυθμό επικοινωνίας.

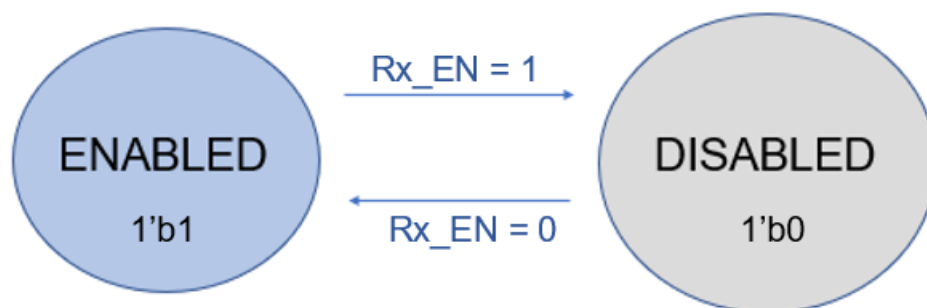
Έπειτα καλείται η συνάρτηση *baud_controller* η οποία μας παρέχει το κατάλληλο ρολόι σύμφωνα με τον προεπιλεγμένο ρυθμό επικοινωνίας. Έτσι κάθε 16 κύκλους του νέου μας ρολογιού δειγματοληπτείται η τιμή του *RxD*. Το πρώτο 0 σηματοδοτεί την έναρξη του μηνύματος, τα επόμενα 8 είναι το μήνυμα προς αποστολή το οποίο και αποθηκεύεται στον καταχωρητή *Rx_DATA* ενώ ταυτόχρονα καταμετρούνται οι άσσοι και ακολουθεί το parity και end bit. Εάν το parity bit είναι σε συμφωνία με τον αριθμό των λογικών 1 που λάβαμε οι

τιμές error1 και error2 παραμένουν αμετάβλητες ενώ στην αντίθετη περίπτωση αυξάνουν την τιμή τους κατά 1 και η μεταβλητή *Rx_PERROR* τίθεται στο **λογικό 1**. Τέλος με το end bit αυξάνεται η τιμή της times.

Με την μετάβαση του σήματος *RX_EN* στο **λογικό 0**, δηλαδή με την απενεργοποίηση του δέκτη ελέγχεται ο αριθμός των δεδομένων που λάβαμε. Εάν αυτός είναι μικρότερος του 11 σημαίνει πως έχει γίνει κακή επιλογή στο *baud_rate* και αυτόματα το σήμα *Rx_FERROR* τίθεται στο **λογικό 1**.

Τέλος, εάν ένα τουλάχιστον από τα σήματα *Rx_PERROR*, *Rx_FERROR* έχει την τιμή 1 η έξοδος τίθεται στο **0** και σηματοδοτεί την λανθασμένη λήψη δεδομένων για οποιονδήποτε από τους δύο λόγους.

Σημειώνεται πως ο UART δέκτης αποτελεί ουσιαστικά μια μηχανή FSM δύο καταστάσεων το σχηματικό της οποίας παρουσιάζεται στην εικόνα 15:



Εικόνα 15: Σχηματικό FSM UART Δέκτη

4) Λειτουργικός Έλεγχος

Για τον λειτουργικό έλεγχο υλοποιήθηκε η συνάρτηση *Meros_B*. Αρχικά δηλώνονται και αρχικοποιούνται βοηθητικές μεταβλητές κάποιες από τις οποίες είναι:

- *Wrs*: για την μέτρηση των φορών που γράφονται δεδομένα στον αποστολέα.
- *Baud_select_false*: για να αποδειχθεί η ορθή λειτουργία του κυκλώματος
- Διάφοροι μετρητές.

Πρωταρχικά, ορίζεται το ρολόι μας με τη ζητούμενη συχνότητα και αρχικοποιούνται οι καταστάσεις των δύο FSM ενεργοποιώντας τον Αποστολέα και κατά συνέπεια τον Δέκτη.

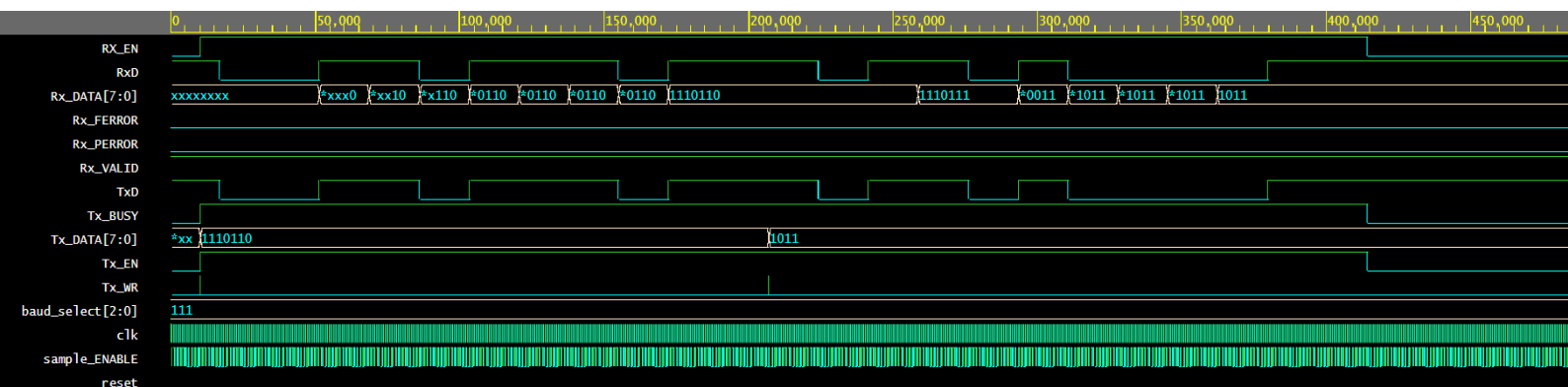
Με την ενεργοποίηση του Αποστολέα και για ένα κύκλο ρολογιού ενεργοποιείται το σήμα *Tx_WR* και ξεκινά η εγγραφή των δεδομένων προς αποστολή στον αποστολέα.

Ξεκινώντας λοιπόν τοποθετούμε τους χαρακτήρες που θέλουμε να στείλουμε σε μια μνήμη message και κάθε φορά που πραγματοποιείται μια αλλαγή στο σήμα *Tx_WR* κωδικοποιείται η πρώτη ή η δεύτερη διάδα/πακέτο αντίστοιχα και τοποθετείται στον καταχωρητή/είσοδο αποστολέα *Tx_DATA*.

Στη συνέχεια θέτουμε το σήμα *RxD* να ακολουθεί το σήμα *TxD* υλοποιώντας έτσι το κανάλι επικοινωνίας μεταξύ των δύο εξαρτημάτων.

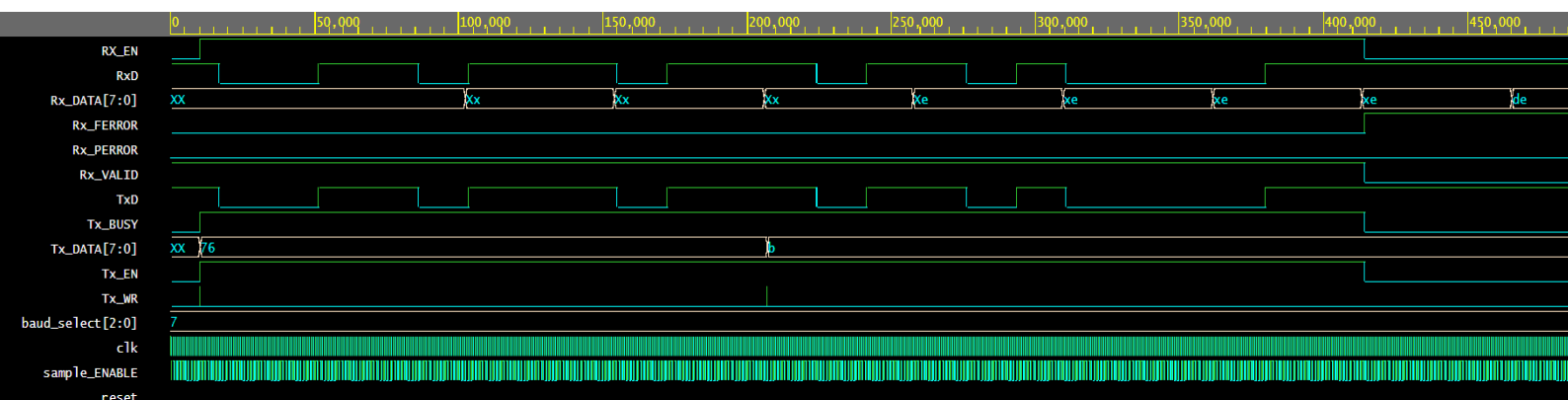
Τέλος ορίζεται πως κατά την 1^η πτώση του σήματος Tx_BUSY στο **λογικό 0** ενεργοποιείται για ένα κύκλο το σήμα σηματοδοτώντας την αποστολή του 2^{ου} πακέτου ενώ με την 2^η πτώση του σήματος στο **λογικό 0** ολοκληρώνεται η μεταφορά όλων των δεδομένων και απενεργοποιούνται και τα 2 εξαρτήματα.

Παρακάτω παρατίθενται εικόνες που επιβεβαιώνουν την ορθή λειτουργία του κυκλώματος, τους χρονισμούς καθώς και τις σχέσεις μεταξύ σημάτων κατά την προβολή του μηνύματος «760f» (εικόνα 16).



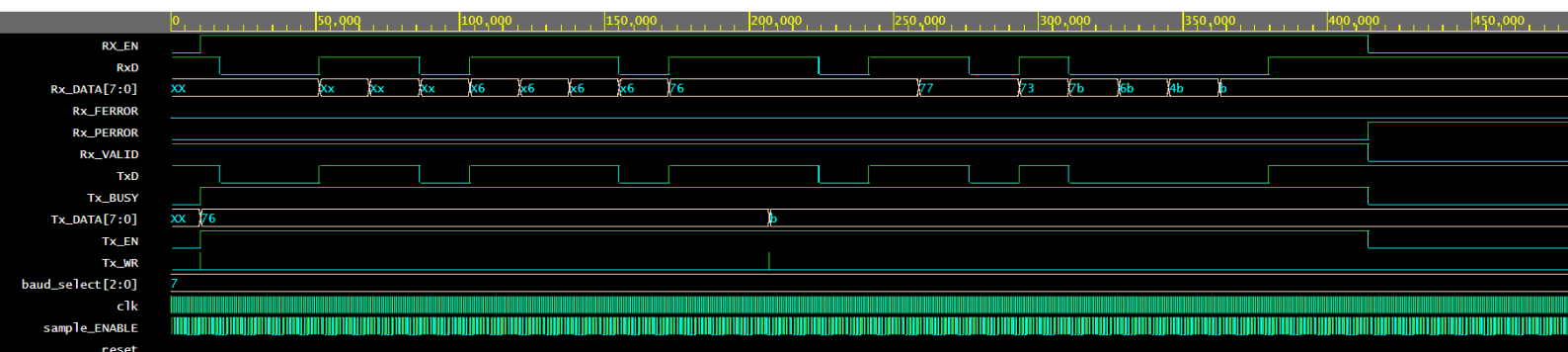
Εικόνα 16: Σύστημα UART Αποστολέας – Δέκτης: Αποστολή του μηνύματος «760f»

Στη συνέχεια χρησιμοποιούνται διαφορετικά baud_rates και τα αποτελέσματα φαίνονται στην εικόνα 17. Παρατηρούμε ότι μετά την απενεργοποίηση των εξαρτημάτων τα σήματα Rx_VALID και Rx_FERROR βρίσκονται στο **λογικό 1** αναπαριστώντας την ύπαρξη σφάλματος.



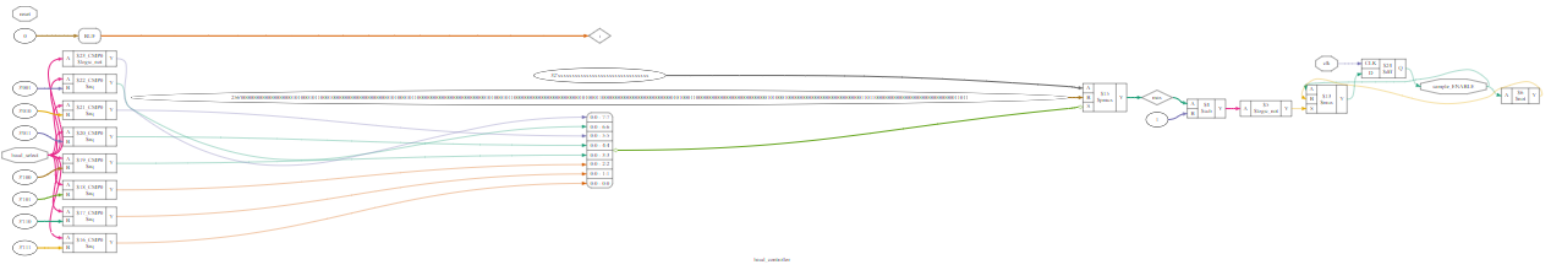
Εικόνα 17: Σύστημα UART Αποστολέας – Δέκτης: Αποστολή του μηνύματος «760f» με διαφορετικά baud_rates

Έπειτα εισάγεται χειροκίνητα σφάλμα στο parity και τα αποτελέσματα φαίνονται στην εικόνα 18.



Εικόνα 18: Σύστημα UART Αποστολέας – Δέκτης: Αποστολή του μηνύματος «760f» με σφάλμα στο parity bit

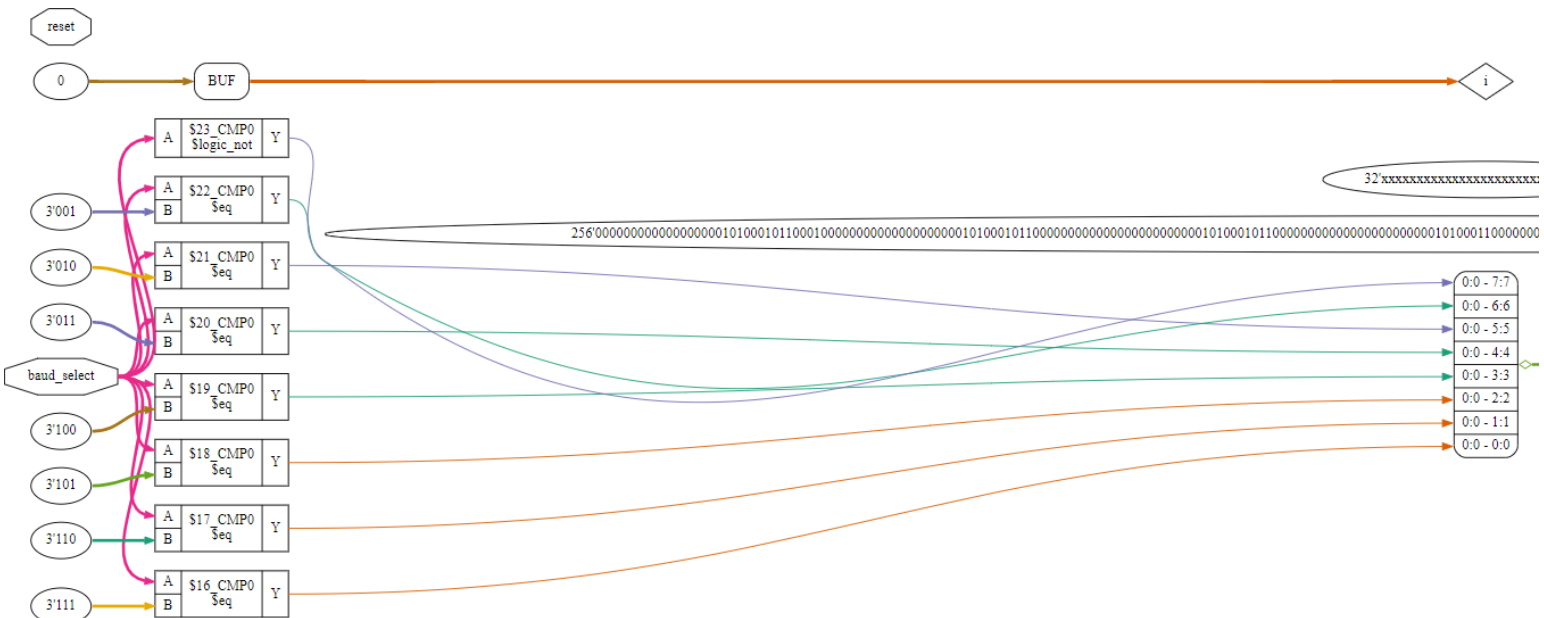
5) Σχηματικά



Εικόνα 19: Σχηματική αναπαράσταση του module baud_controller.

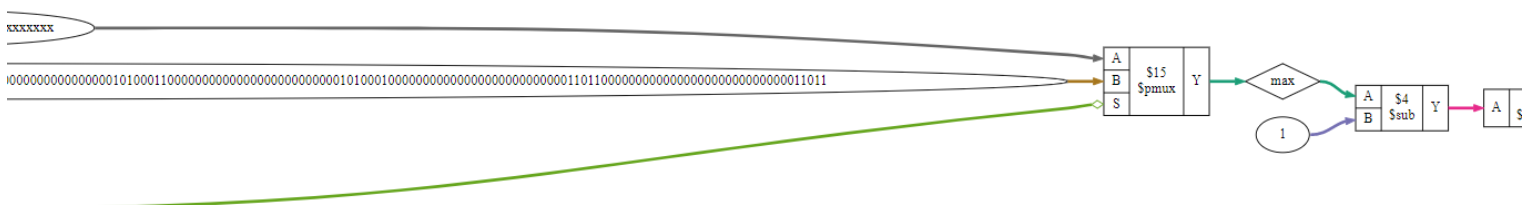
Συγκεκριμένα αν κάνουμε zoom σε κάθε κομμάτι έτσι ώστε να είναι πιο καθαρή εικόνα προκύπτουν 3 μέρη

Το 1^ο μέρος είναι στα αριστερά



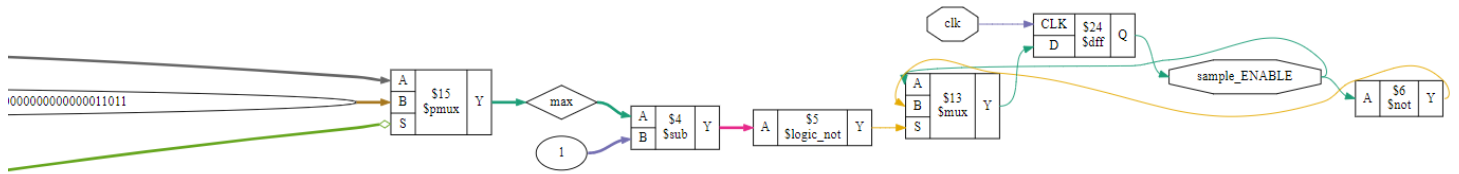
Εικόνα 20: Σχηματική αναπαράσταση του module baud_controller (zoom left).

Το 2^ο μέρος είναι η εικόνα στο κέντρο

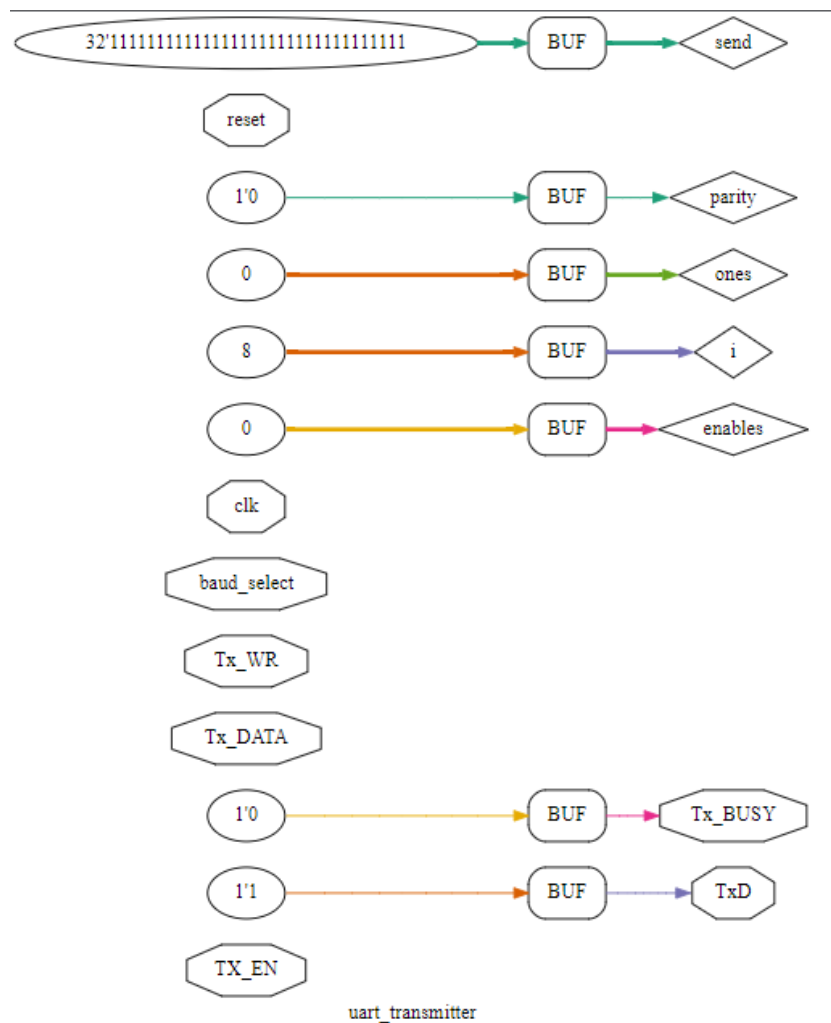


Εικόνα 21: Σχηματική αναπαράσταση του module baud_controller (zoom center).

Το 3^ο μέρος είναι η εικόνα στα αριστερά



Εικόνα 22: Σχηματική αναπαράσταση του module baud_controller (zoom right).

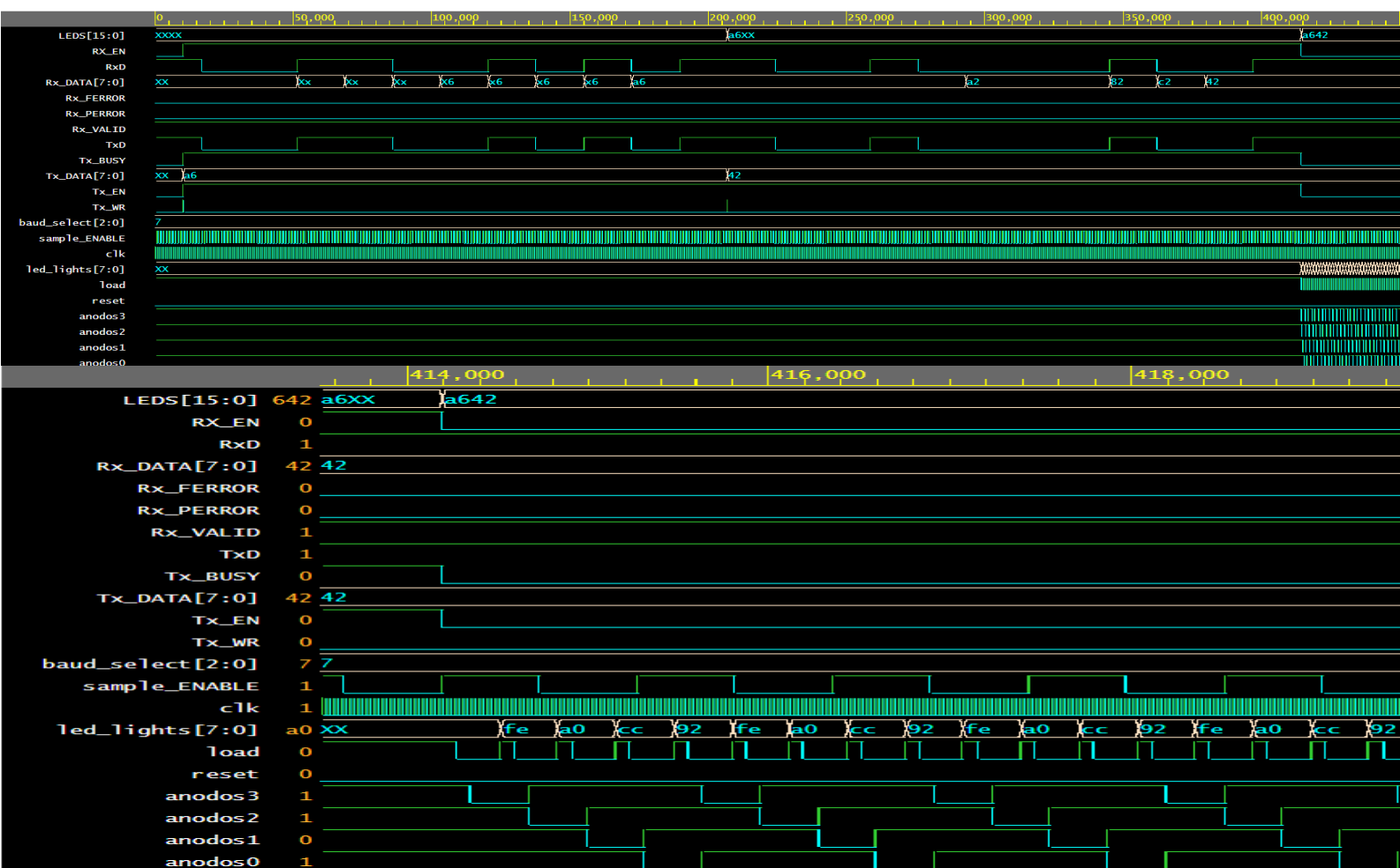


Εικόνα 23: Σχηματική αναπαράσταση του module transmitter.

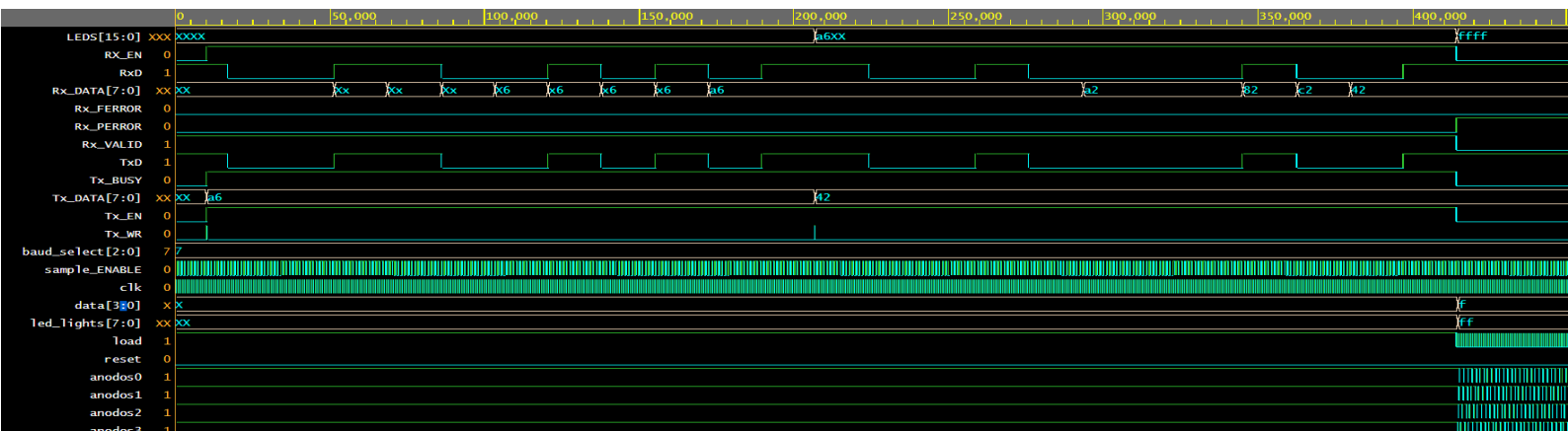
Σημειώνεται πως αντιμετωπίσαμε ένα πρόβλημα με την εξαγωγή του σχηματικού του Δέκτη και υποθέτουμε πως ευθύνεται η ύπαρξη πολλών sensitivity list.

ΜΕΡΟΣ Γ' – ΣΥΝΕΝΩΣΗ

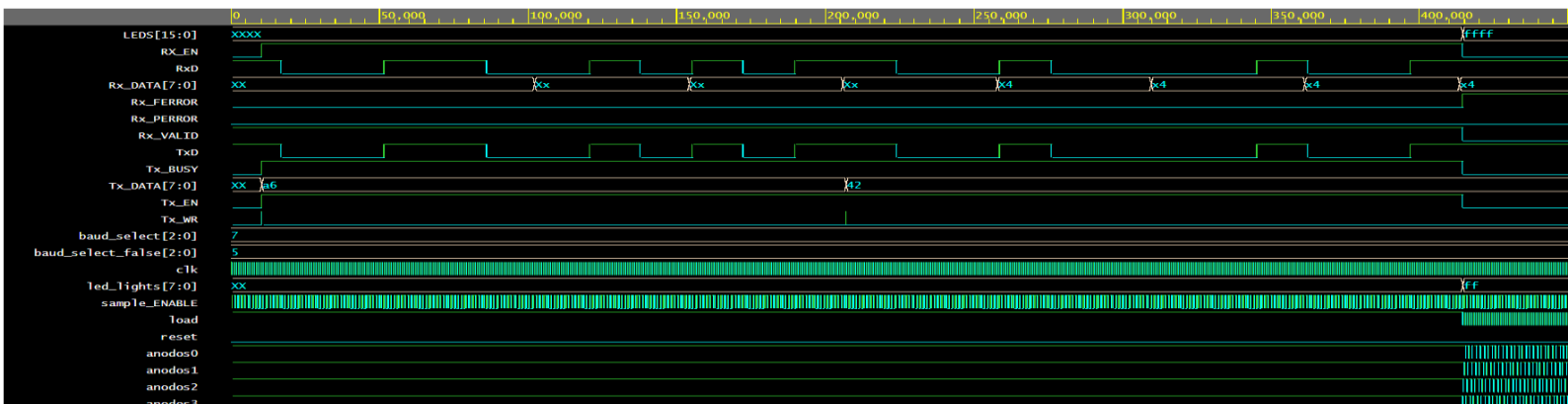
Στο τρίτο μέρος της εργασίας ενώθηκαν κατάλληλα τα μέρη Α και Β με στόχο την προβολή των δεδομένων που αποστάλθηκαν από τον πομπό και στη συνέχεια λήφθηκαν από τον δέκτη, στις τέσσερις οθόνες LED. Για την ορθή λειτουργία προστέθηκε ως έξοδος του UART δέκτη ένας καταχωρητής 16 θέσεων **LEDS[15:0]** για την αποθήκευση και των 2 πακέτων που στέλνει ο αποστολέας. Έτσι την πρώτη φορά που τίθεται σε λειτουργία ο δέκτης κατά τη δειγματοληψία αποθηκεύει τα δεδομένα όπως αυτά βρίσκονται στον καταχωρητή Rx_DATA στις τελευταίες 8 θέσεις του καταχωρητή LEDS ενώ κατά την δειγματοληψία του δεύτερου πακέτου αποθηκεύει τα δεδομένα στις πρώτες 8 θέσεις. Αν διαπιστωθεί πως υπάρχει σφάλμα κατά την δειγματοληψία είτε στο parity είτε έχει χρησιμοποιηθεί διαφορετικός ρυθμός επικοινωνίας μεταξύ πομπού και δέκτη, ο καταχωρητής παίρνει την τιμή 'ffff'. Αφού λοιπόν ολοκληρωθεί η αποστολή και η λήψη ολόκληρου του μηνύματος στέλνεται σήμα ενεργοποίησης στον counter του μέρους Α με σκοπό την παραγωγή των σημάτων load και anodoi για την φόρτωση και προβολή του μηνύματος στις 4 οθόνες. Σε κάθε θετική ακμή του load εισχωρείται στον καταχωρητή **data[3:0]** μια τετράδα bits από τον καταχωρητή LEDS ξεκινώντας από την [15:12] και καταλήγοντας στην [3:0], η οποία αποτελεί είσοδο στην συνάρτηση αποκωδικοποίησης του μέρους Α. Τα αποτελέσματα της παραπάνω διαδικασίας φαίνονται στις εικόνες 24,25,26,27.



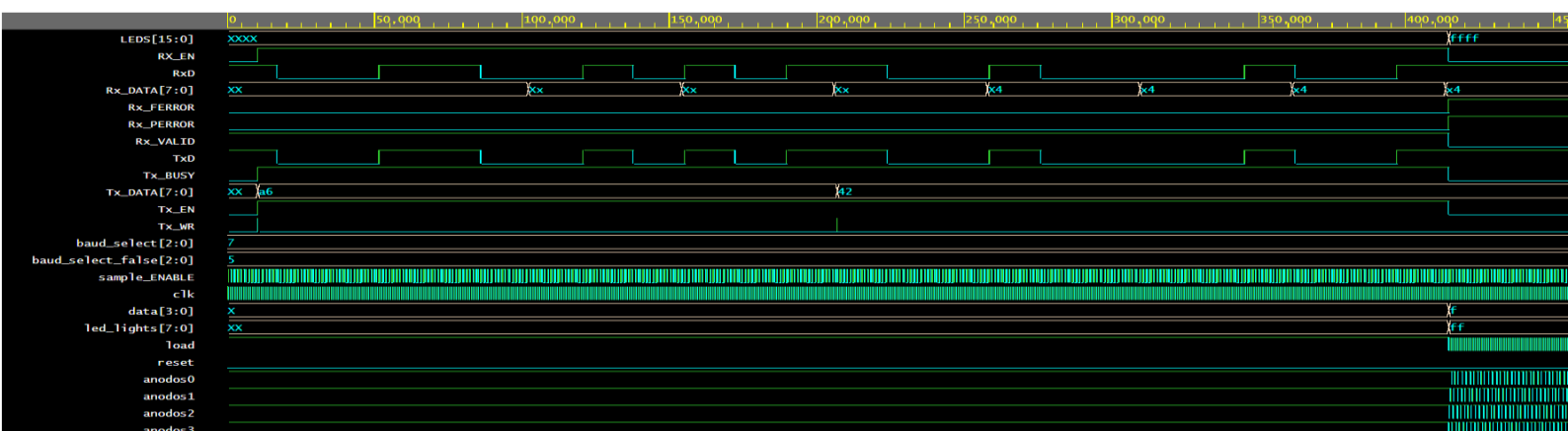
Εικόνα 24: Σύστημα UART Αποστολέας – Δέκτης-LEDS: Αποστολή του μηνύματος «-642»



Εικόνα 25: Σύστημα UART Αποστολέας – Δέκτης: Αποστολή του μηνύματος «-642» με σφάλμα στο parity bit



Εικόνα 26: Σύστημα UART Αποστολέας – Δέκτης: Αποστολή του μηνύματος «-642» με σφάλμα στον ρυθμό επικοινωνίας



Εικόνα 27: Σύστημα UART Αποστολέας – Δέκτης: Αποστολή του μηνύματος «-642» με σφάλμα στο parity bit και στον ρυθμό επικοινωνίας

ΜΕΡΟΣ Δ' – ΚΩΔΙΚΟΠΟΙΗΤΗΣ / ΑΠΟΚΩΔΙΚΟΠΟΙΗΤΗΣ

Στο τέταρτο μέρος υλοποιήθηκε η κωδικοποίηση που συμβαίνει στον αποστολέα πριν στείλει το μήνυμα και η αντίστοιχη αποκωδικοποίηση που χρειάζεται να κάνει ο δέκτης έτσι ώστε μόλις λάβει το κωδικοποιημένο μήνυμα, να μπορέσει να το ανακτήσει. Καθώς τα bits που στέλνονται είναι μη προσημασμένα (unsigned), δεν μπορούσε να χρησιμοποιηθεί κωδικοποίηση που μετατρέπει κάποια bit σε -1, οπότε επιλέχθηκε η κωδικοποίηση Gray. Η συγκεκριμένη κωδικοποίηση υπερτερεί απέναντι στις άλλες, διότι χαρακτηρίζεται από ομαλές μεταβάσεις μεταξύ γειτονικών συμβόλων και λιγότερες απαιτήσεις hardware σε σχέση με άλλες κωδικοποιήσεις.

1) Κωδικοποιητής Gray

Η συνάρτηση encoder δέχεται ένα όρισμα, το Tx_DATA, που είναι η ακολουθία 8 bit που καλείται ο αποστολέας να στείλει. Σκοπός είναι το output , gray_Data, που είναι καταχωρητής των 8 bit, να είναι το κωδικοποιημένο πλέον μήνυμα. Αυτό πραγματοποιείται με τη χρήση πυλών XOR. Συγκεκριμένα, αφού τα δεδομένα κωδικοποιούνται παράλληλα, ξεκινάμε από το 8ο bit του gray_Data, που του δίνεται ακριβώς η τιμή του αντίστοιχου bit από το Tx_DATA. Συνεχίζουμε δίνοντας σε κάθε επόμενο στοιχείο του output gray_Data, την τιμή που προκύπτει από την πύλη XOR, με εισόδους το αντίστοιχο στοιχείο του input, και του προηγούμενου στοιχείου gray_Data που κωδικοποιήθηκε.

Η συνάρτηση uart_transmitter, όπως ακριβώς υλοποιήθηκε στο 3^ο μέρος, καλεί τη συνάρτηση του κωδικοποιητή encoder, δίνοντας της σαν input, το αντίστοιχο δικό της input από την ακολουθία 8 bit που καλείται να στείλει έτσι ώστε να το κωδικοποιήσει. Παίρνει τα κωδικοποιημένα bits που προκύπτουν από τον encoder και δουλεύει πάνω σε αυτά. Θέτει τιμή στο parity bit, μετρώντας τα στοιχεία των gray_Data που έχουν την τιμή 1, όπως ακριβώς συνέβαινε και πριν. Δίνει κάθε φορά στο output TxD, με τον ρυθμό που προκύπτει από το baud_rate, την τιμή από τα στοιχεία του gray_Data, σειριακά. Συνεπώς, ο αποστολέας λειτουργεί ακριβώς με τον ίδιο τρόπο, με τη μόνη διαφορά, ότι επεξεργάζεται τα κωδικοποιημένα δεδομένα.

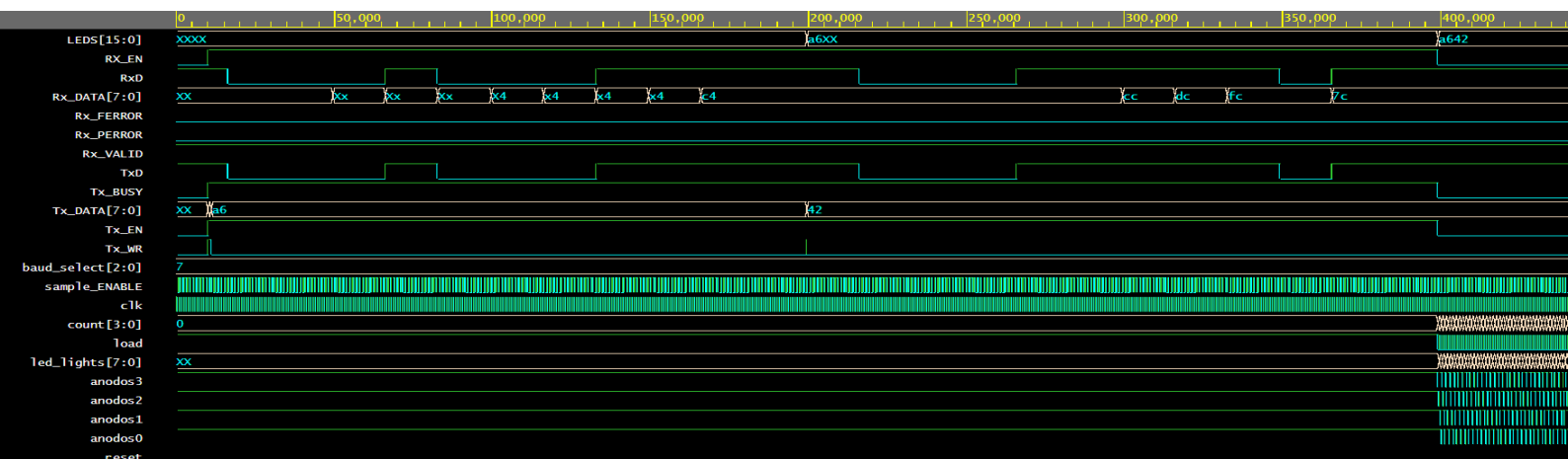
2) Αποκωδικοποιητής Gray

Η συνάρτηση decoder, δέχεται ένα όρισμα, το gray που είναι η ακολουθία 8 bit των κωδικοποιημένων δεδομένων που έχει λάβει ο δέκτης από τον αποστολέα. Αποκωδικοποιούνται παράλληλα, οπότε έχουν ένα output, το data, που είναι ο καταχωρητής 8 bit των αποκωδικοποιημένων δεδομένων, δηλαδή το τελικό μήνυμα που λαμβάνεται. Η αποκωδικοποίηση υλοποιείται με τη χρήση πυλών XOR. Συγκεκριμένα, δίνεται στο 8^ο bit των data ακριβώς το αντίστοιχο από τα gray. Στη συνέχεια, δίνεται στα στοιχεία του output data, η τιμή που προκύπτει κάθε φορά από πύλη XOR, με εισόδους αντίστοιχο το στοιχείο του gray και το προηγούμενο. Παρατηρείται ότι όντως με αυτόν τον τρόπο, το μήνυμα που προκύπτει είναι ίδιο με το μήνυμα που δόθηκε στον κωδικοποιητή.

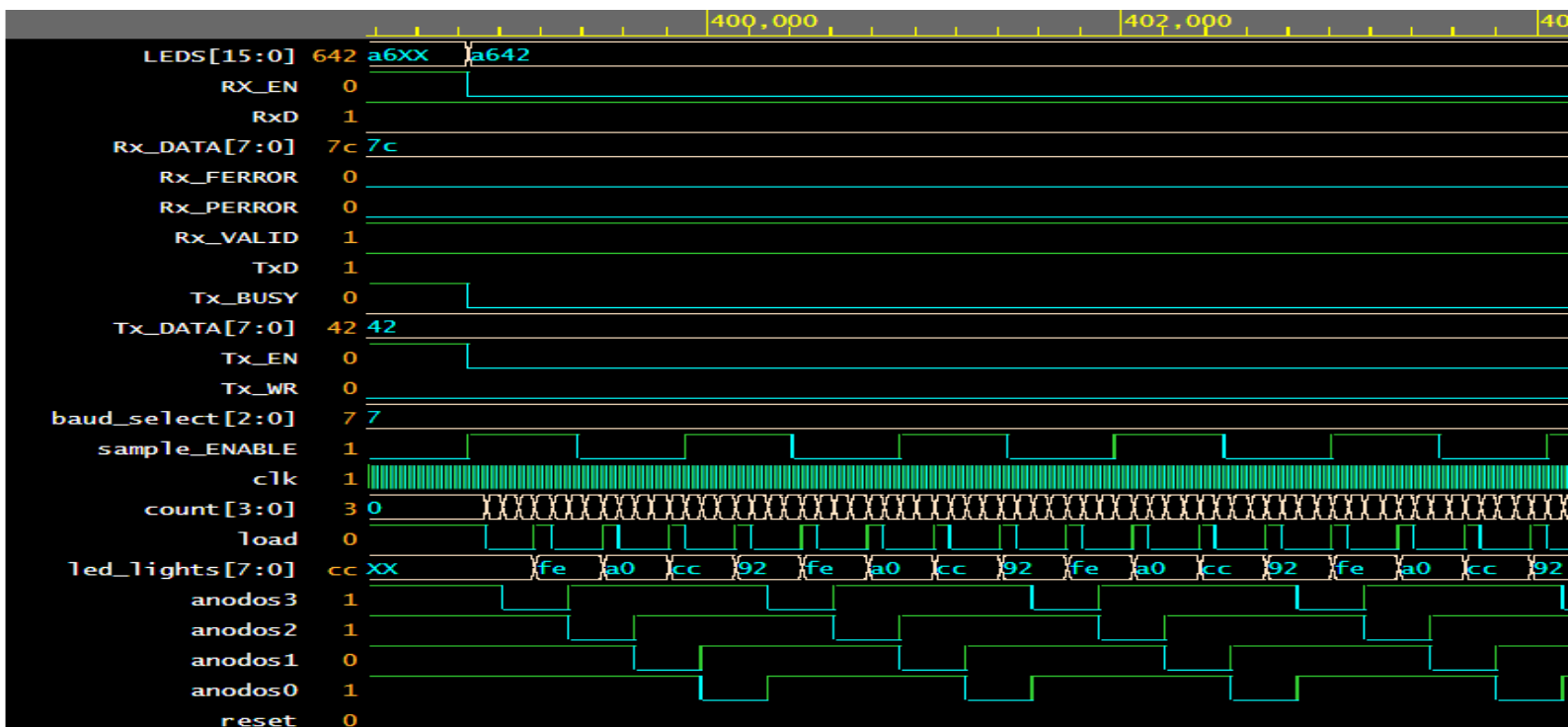
Η συνάρτηση *uart_receiver*, δέχεται τα δεδομένα της σειριακά. Οπότε, καλεί τη συνάρτηση decoder και της δίνει σαν input την ακολουθία Rx_DATA που έχει κανονικά ο δέκτης σαν output. Προκύπτει τη στιγμή που έρχονται όλα τα δεδομένα από τον αποστολέα και συμβαίνει αυτό, έτσι ώστε τα δεδομένα Rx_DATA, μόλις είναι έτοιμα στον δέκτη, να περάσουν από το τελικό στάδιο της αποκωδικοποίησης. Έτσι τα δεδομένα data που είναι το output της decoder, είναι το μήνυμα που έχει καταφέρει να ανακτήσει ο δέκτης.

3) Λειτουργικός Έλεγχος

Αρχικά, δίνονται οι κυματομορφές που προκύπτουν από φυσιολογική λειτουργία, χωρίς κάποιο error.

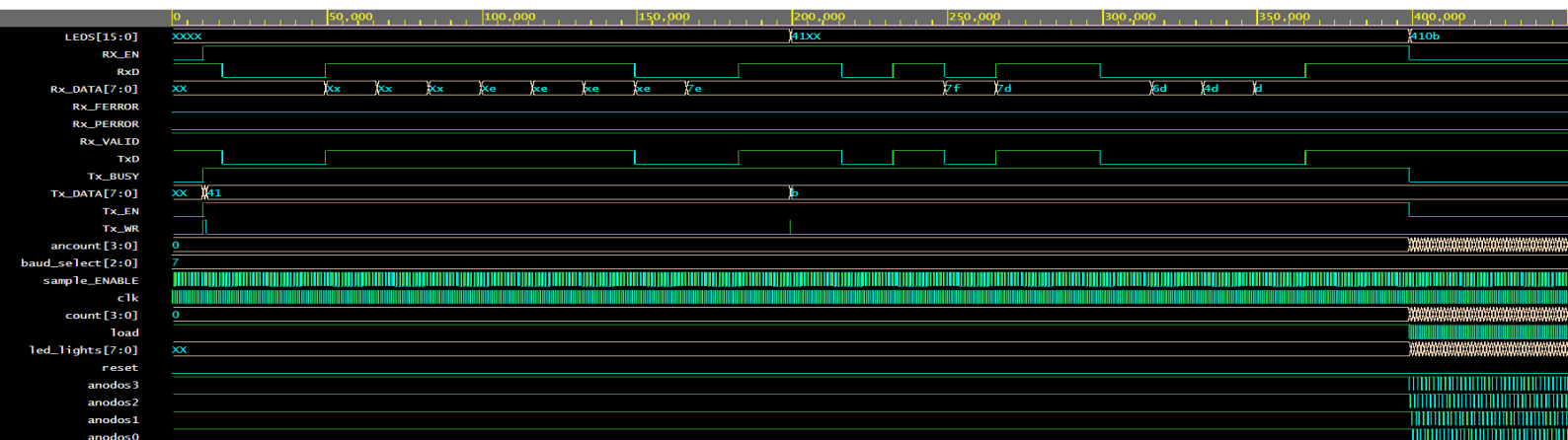


Εικόνα 28: Σύστημα UART αποστολέα/δέκτη με τη χρήση κωδικοποιητή/αποκωδικοποιητή Gray (Αποστολή μηνύματος “-642”)



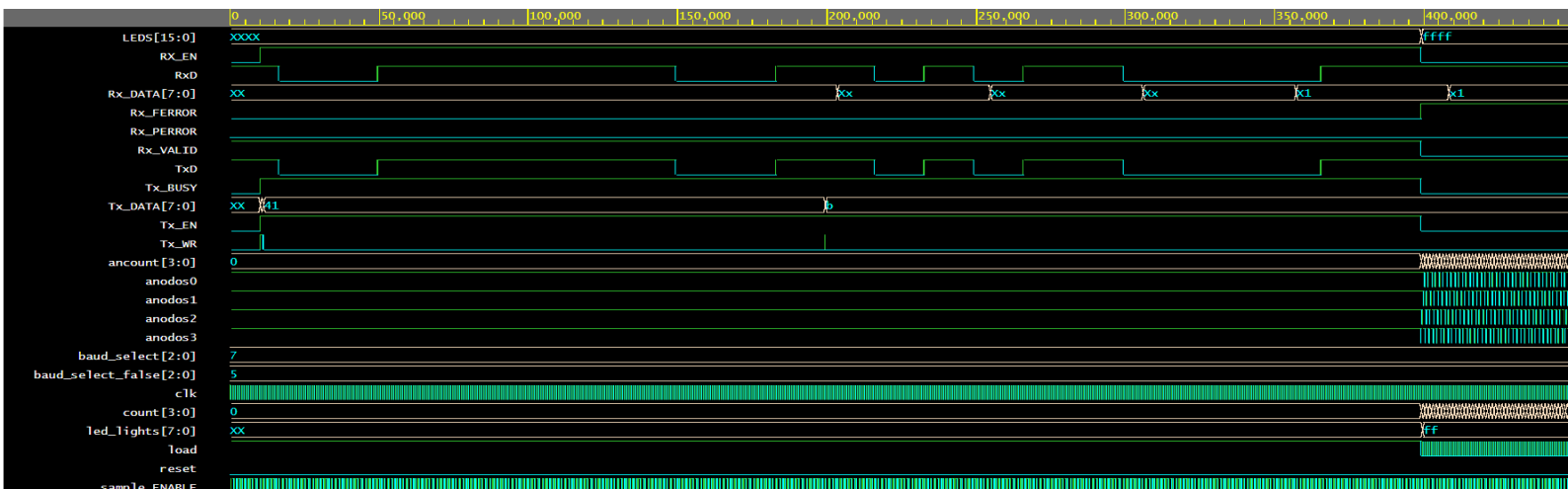
Εικόνα 29: Ομαλή λειτουργία της υλοποίησης του μέρους Δ, με την προσθήκη κωδικοποιητή/αποκωδικοποιητή Gray

Φαίνεται πως όλα λειτουργούν ομαλά ακριβώς όπως το μέρος Γ και ότι η κωδικοποίηση/αποκωδικοποίηση λειτουργεί σωστά. Συγκεκριμένα, η πρώτη ακολουθία που στέλνεται είναι το -6, όπως δείχνει το *Tx_DATA*. Το *Rx_DATA*, έχει το κωδικοποιημένο μήνυμα που λαμβάνει και έπειτα, όπως είναι σχεδιασμένο περνάει από το στάδιο της αποκωδικοποίησης και καταλήγει στον καταχωρητή *LEDS*. Φαίνεται ότι όντως ο καταχωρητής *LEDS*, λαμβάνει το μήνυμα -6. Στη συνέχεια στέλνεται το μήνυμα 42, όπως δείχνει *Tx_DATA*. Με τον ίδιο τρόπο, στο *Rx_DATA* εμφανίζεται το κωδικοποιημένο μήνυμα, το οποίο περνάει από τον decoder και στο τέλος ο *LEDS* αποθηκεύει όντως το μήνυμα 42.



Εικόνα 30: Σύστημα UART αποστολέα/δέκτη με τη χρήση κωδικοποιητή/αποκωδικοποιητή Gray (Αποστολή μηνύματος “410f”)

Άλλο ένα παράδειγμά με διαφορετικό μήνυμα. Συμβαίνει η ίδια διαδικασία. Ο καταχωρητής Leds αποθηκεύει τις αποκωδικοποιημένες ακολουθίες που στέλνει ο πομπός (Tx_DATA), παρόλο που εμφανίζονται στον δέκτη (Rx_DATA) κωδικοποιημένα.



Εικόνα 31: Σύστημα με λάθος ρυθμό (Rx_ERROR) και χρήση κωδικοποιητή/αποκωδικοποιητή Gray)

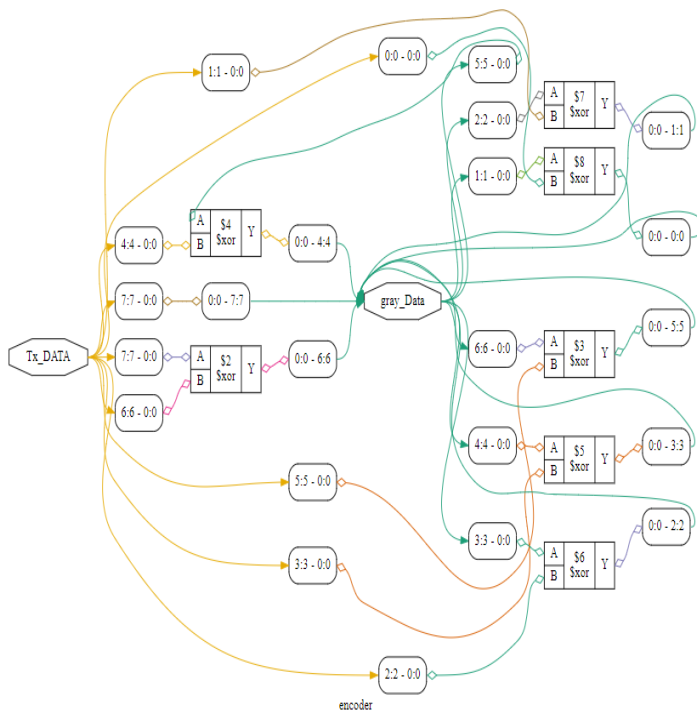
Δίνεται αυτή η εικόνα για να φανεί ότι σε περίπτωση σφάλματος, το σύστημα λειτουργεί ακριβώς όπως θα έπρεπε.

	0	50	100	150	200
data[7:0]	57		7b		d0
gray[7:0]	65		52		9f
gray_Data[7:0]	65		52		9f
Tx_DATA[7:0]	57		7b		d0

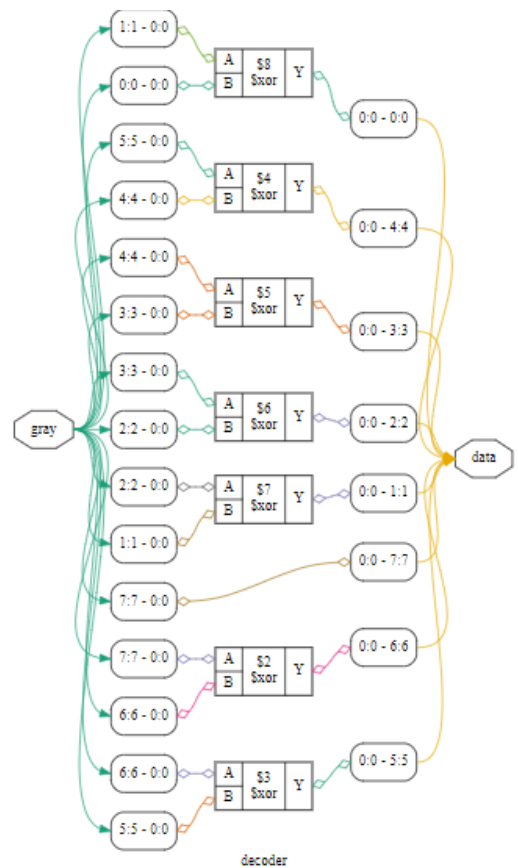
Εικόνα 32: Σύστημα κωδικοποιητή/ αποκωδικοποιητή Gray (μεμονωμένα, χωρίς αποστολέα/ δέκτη και 7segment) Αποστολή μηνυμάτων «57, 7b, d0»

Εδώ δοκιμάζεται η λειτουργία του κωδικοποιητή και αποκωδικοποιητή Gray χωρίς κάποιο άλλο σύστημα. Αρχικά στέλνεται το μήνυμα 57, μετά το 7b και τέλος το d0. Φαίνονται όλα τα στάδια από τα οποία περνάνε τα δεδομένα στις συναρτήσεις encoder και decoder και πως στο τέλος το μήνυμα ανακτάται.

4) Σχηματικά



Εικόνα 33: Σχηματική αναπαράσταση του encoder



Εικόνα 34: Σχηματική αναπαράσταση του decoder

ΣΥΝΔΕΣΜΟΙ EDAPLAYGROUND

Μέρος Α:

<https://edaplayground.com/x/NRys>

Μέρος Β:

<https://edaplayground.com/x/Hy8Y>

Μέρος Γ:

<https://edaplayground.com/x/rAsG>

Μέρος Δ:

<https://edaplayground.com/x/9NHV>

ΣΤΟΙΧΕΙΑ ΦΟΙΤΗΤΩΝ

- Γκατζή Κωνσταντίνα
Α.Ε.Μ.: 10037
Email: gikonstan@ece.auth.gr
- Παπακώστας Κωνσταντίνος
Α.Ε.Μ.: 10127
Email: kdpapako@ece.auth.gr